Implementation Patterns

The high-level design of system components

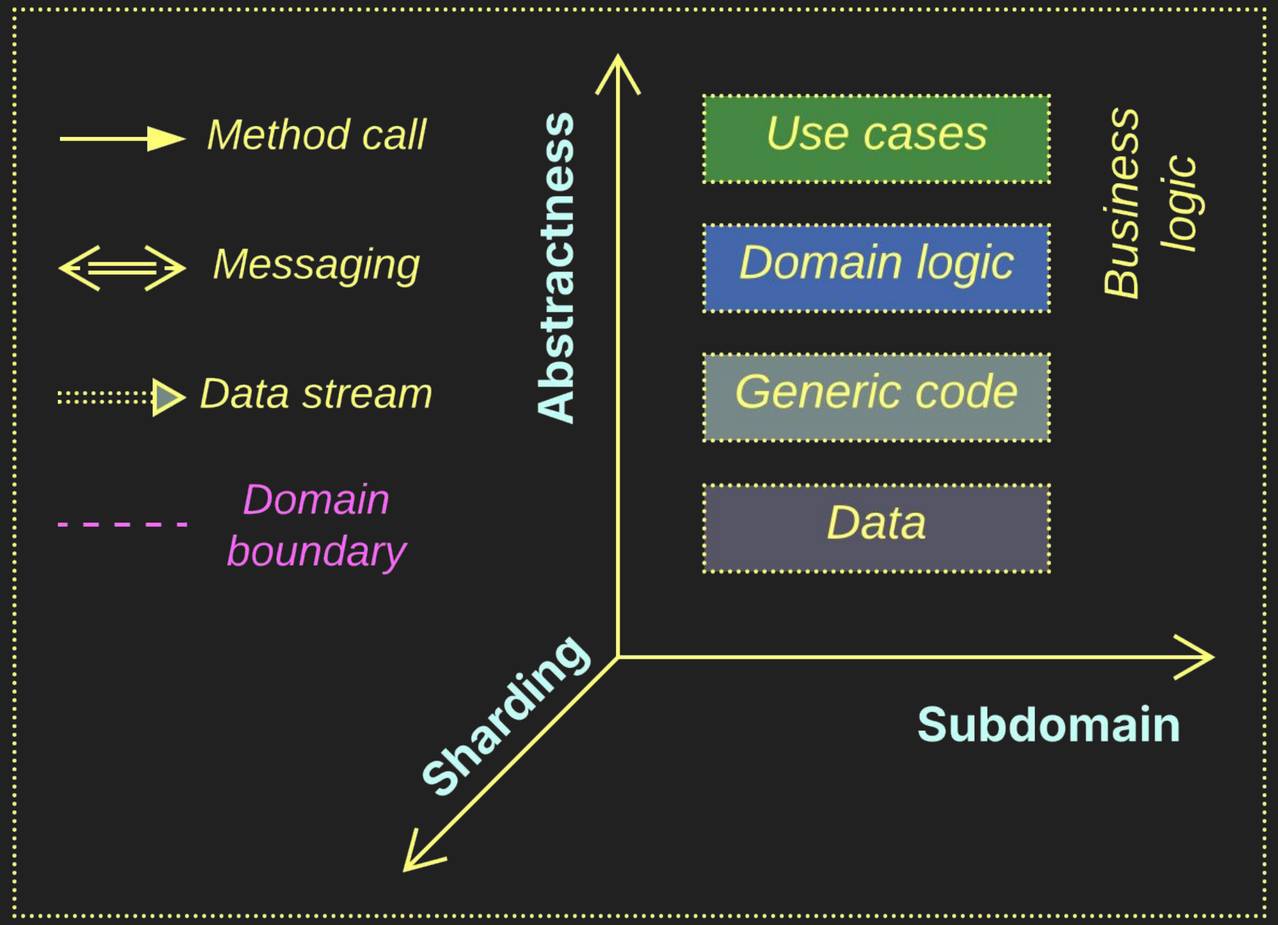
Contents

Several patterns are found inside system components:

- *Plugins* customize the component's behavior.
- *Hexagonal Architecture* isolates the business logic from external dependencies.
- *Microkernel* mediates between resource providers and resource consumers.
- *Mesh* maintains a decentralized system.

The system of coordinates

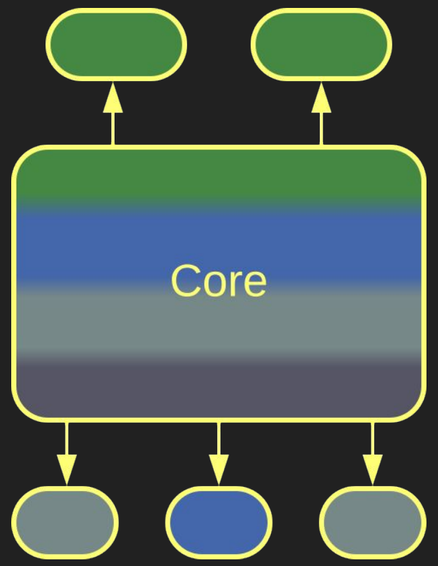
All architectures are color-coded and drawn in the same system of coordinates



Plugins

Customized behavior

Plugins – overview



Plugins

If you need your component to be customizable, you have it support *plugins* – replaceable subcomponents that define key aspects of your *core* behavior.

Plugins is a loosely defined pattern with many variations.

Plugins power product lines and frameworks.

Heavy use of *plugins* may degrade performance.

Plugins – trade-offs

Select aspects of your system's behavior become customizable

Sometimes the custom behavior can be defined in a high-level DSL

Plugins may employ platform-specific optimizations

It's hard to deduce good plugin APIs

Plugins impede whole-system performance optimization

Testability is poor as there are too many possible configurations

Plugins – integration

Plugins may involve components that use the system or are used by it:

- True *Plugins* are registered with the system's *core* and are called by the *core*. Each plugin has a single function in the system and is usually a third party product.
- *Add-ins* come from the component's authors and have access to its internals. They are like optional but deeply integrated modules that often feature business logic.
- *Extensions* modify business logic by changing existing or adding new use cases but are still called and managed by the *core*.
- *Addons* build a layer of indirection on top of the system's public API.

Plugins – abstractness

Plugins may be more or less abstract than the system's *core*:

- High-level *plugins* or *addons* customize user experience or collect metadata. They may run independently and use the system's API.
- Low-level *plugins* encapsulate algorithms, business rules, or hardware acceleration. They are called by the *core* as a part of its workflow.
- A customization may involve a set of both high- and low-level *plugins*.

Plugins – role

A *plugin* may:

- Provide input from a UI widget or CLI connection to the *core*.
- Collect output, such as statistics or logs, from the *core*.
- Participate in both input and output. Health check belongs here.
- Behave as a *Controller* by receiving input from the *core* and deciding on the further *core*'s actions.
- Be a data processor for the *core* by implementing a step of its main algorithm. Examples include codecs and the use of SIMD / DSP / cryptoprocessors.

Plugins – linkage

Plugins may be selected at different stages:

- *Flavors* are special builds of a software with functionality customized through different sets of *plugins*. A more functional *flavor* is more expensive.
- An application may choose *plugins* at startup according to its configuration file.
- Some software systems allow for changing *plugins* at runtime.

Plugins – granularity

A *plugin* can be large or tiny:

- A small function or class built into the application corresponds to the *Strategy* pattern.
- An *aspect* implements some kind of pervasive functionality, such as logging.
- An external component (library, application or remote service) may be used by the *core* as a *plugin*.

Plugins – multiplicity

A *plugin* can be optional or mandatory:

- A *mandatory plugin*, such as an implementation of an algorithm or business rule, is required for the proper functioning of the core.
- An *optional plugin* may be absent, in which case the core substitutes an empty or trivial implementation. This is common with logging or statistics.
- It can also be *subscriptional*, in which case multiple instances of an *optional plugin* are allowed. This helps if you want your logs to be both saved to the local file system and sent over the network.

Plugins – execution

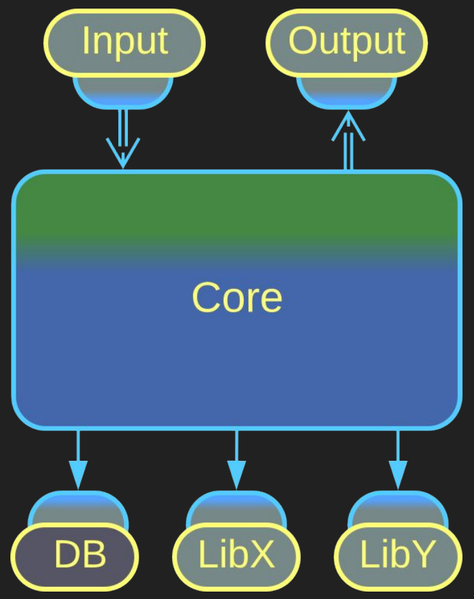
A *plugin* can be executed in a variety of ways:

- It can be a selectable part of the *core*, or a statically or dynamically linked library.
- It can be written in a *domain-specific language* (*DSL*) which is interpreted or compiled by the *core*.
- It can reside on a remote server or in a hardware device accessible via RPC or messaging.

Hexagonal Architecture

Isolation from dependencies

Hexagonal Architecture – overview



Hexagonal Architecture

Hexagonal Architecture is a variation of *Plugins* designed to isolate business logic from dependencies on external components.

Anything not belonging to the *core* logic is hidden behind an *adapter* which translates between the SPI of the core and the API of the plugged-in component.

Hexagonal Architecture is mandatory for long-lived products.

It does not benefit small components or proofs of concept.

Hexagonal Architecture – trade-offs

Protects the business logic from external influences

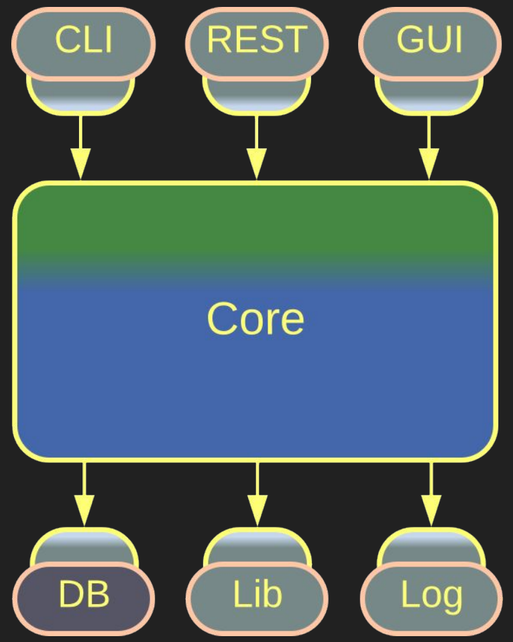
Protects business logic programmers from the need to learn technologies

Facilitates the use of stubs / mocks

The performance is suboptimal

Extra APIs need to be designed and maintained

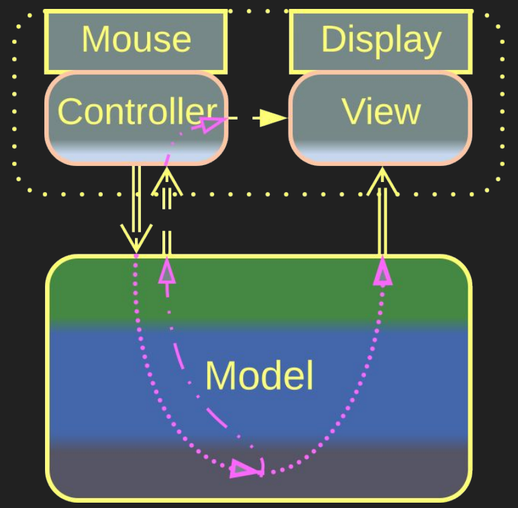
Hexagonal Architecture – variants



Hexagonal Architecture

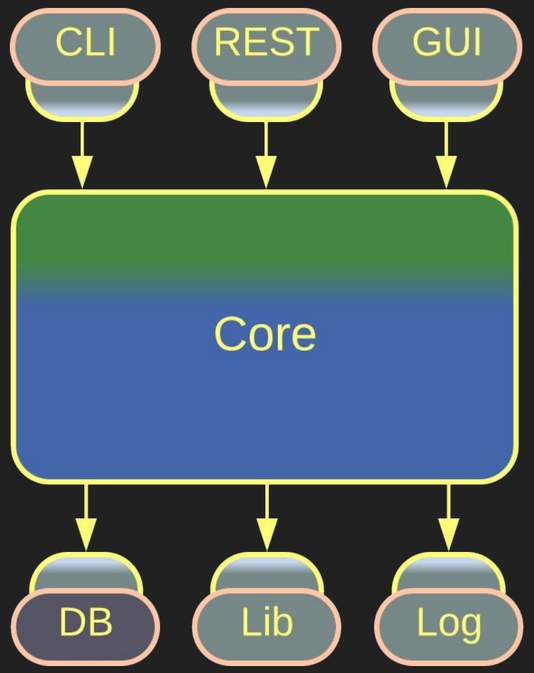
The true *Hexagonal Architecture* isolates the application's business logic from all its dependencies.

Its older sibling, *Separated Presentation*, decouples the core application from its user interface.



Separated Presentation

Hexagonal Architecture – examples



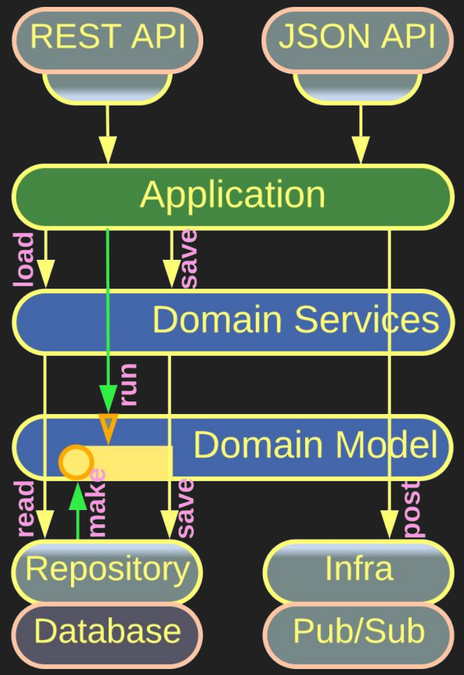
Ports and Adapters, Hexagonal Architecture

Ports and Adapters is the original *Hexagonal Architecture*. It inserts *adapters* between the business logic *core* and any *external components*, such as third party services, libraries or databases.

If the interface of an external component changes, or even the whole component needs to be replaced by an incompatible one from another vendor, its *adapter* absorbs the change, leaving the business logic intact.

As the result, the business logic lives on its own terms, while anything outside of it is dispensable.

Hexagonal Architecture – examples

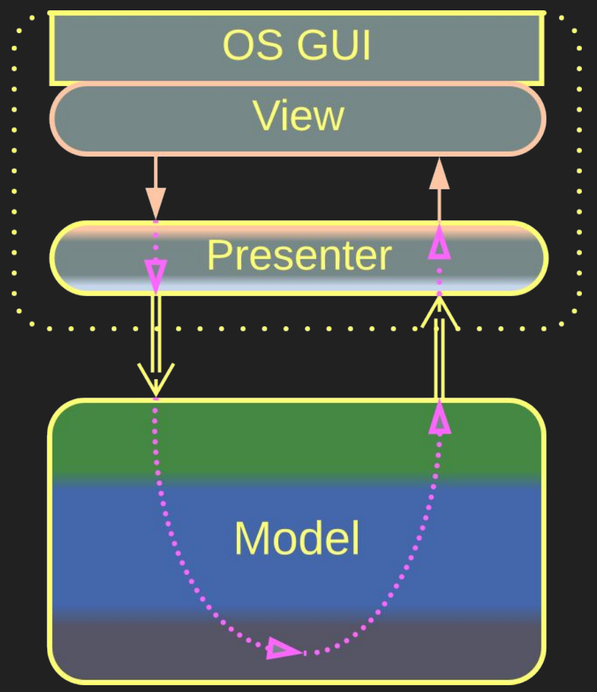


Onion Architecture,
Clean Architecture

Onion Architecture delves into the structure of the *Hexagonal Architecture*'s *core*, dividing it into *Layers* by the canons of *Domain-Driven Design*.

Clean Architecture is a later generalization or redesign of *Onion Architecture*.

Separated Presentation – examples

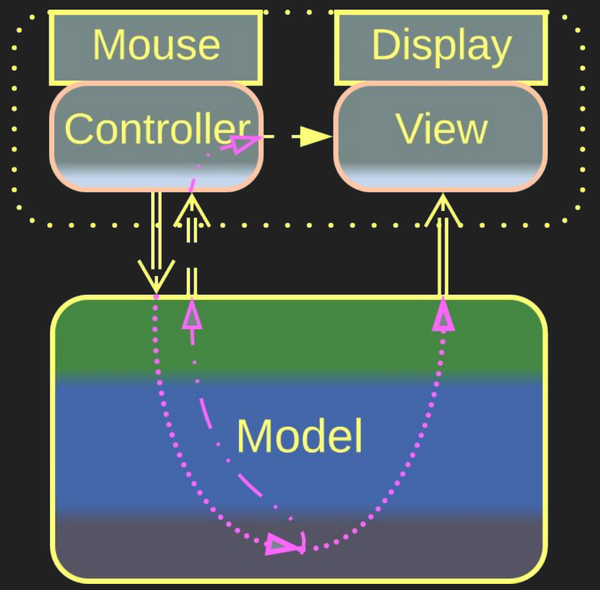


Model-View-Presenter

Model-View-Presenter (*MVP*) and the related *MVA*, *MVVM*, *Model 1*, and *Document-View* insert one or two layers of indirection between the main application, called *model*, and its user interface, usually built with an OS GUI or a web framework.

The indirection allows for replacing or editing the UI without affecting the core, supporting cross-platform development and look and feel upgrades.

Separated Presentation – examples



Model-View-Controller

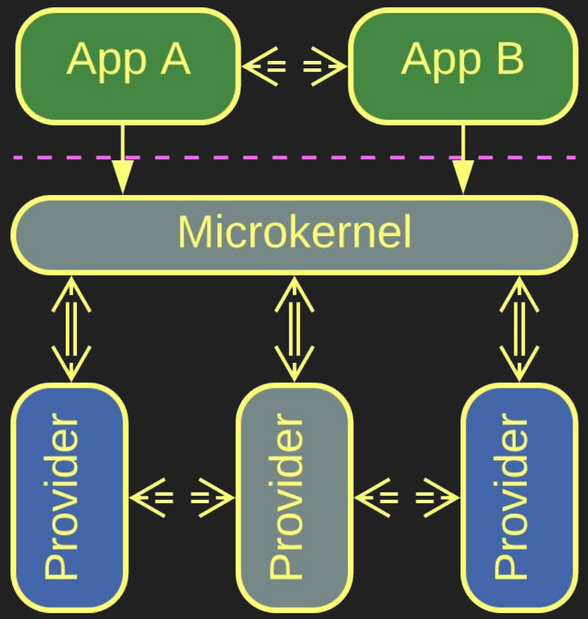
Model-View-Controller (*MVC*) and the related *ADR*, *RMR*, and *Model 2* separate the model's input from its output. These patterns process raw mouse clicks (or HTTP requests) and draw home-made windows (or send XML responses).

This makes sense when well-established GUI or web frameworks are unavailable or don't match the system's requirements.

Microkernel

Management of shared resources

Microkernel – overview



Microkernel

Microkernel is another variation of *Plugins* with a rudimentary *core*, called *microkernel*, which mediates between *resource providers* and *resource consumers* (applications).

The *microkernel* is a *Middleware* for the *applications* and an *Orchestrator* for the *resource providers*.

Microkernel is found in frameworks with complex behavior.

It is ill-suited if the *providers* are coupled to each other.

Microkernel – trade-offs

Microkernel tackles system complexity by distributing it across multiple roles

Polymorphic components, often from unknown vendors, are supported

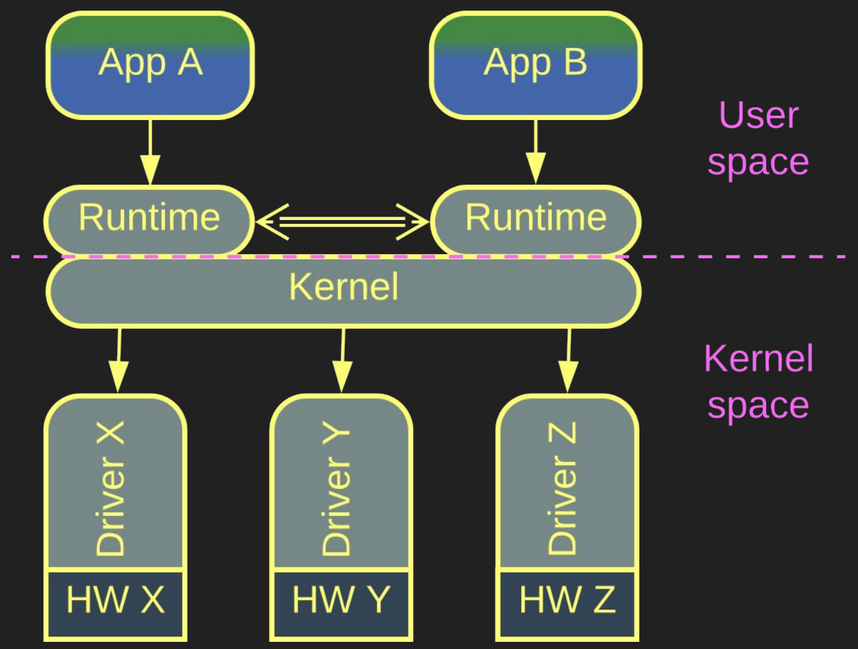
The applications are sandboxed and platform-agnostic

Microkernel's APIs and SPIs are hard to change after the initial release

Indirection degrades performance

The latency is unpredictable because of resource contention

Microkernel – examples

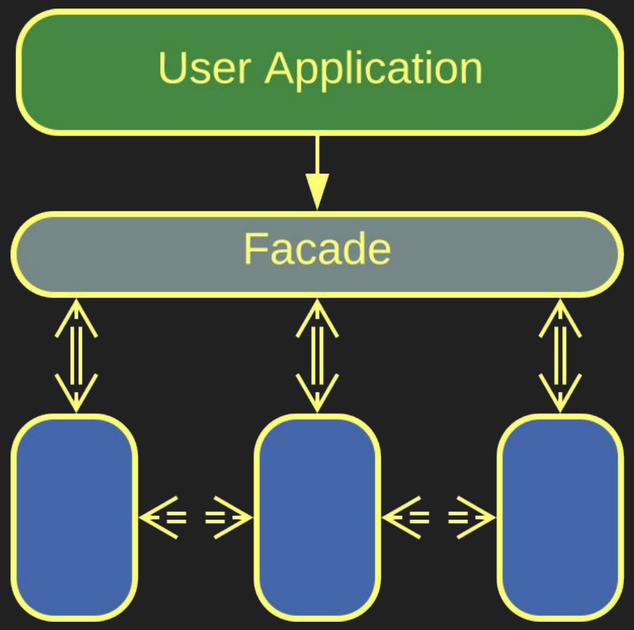


This is the original inspiration for *Microkernel*.

Though not “micro-”, the kernel of an operating system is exactly this thing – it sandboxes user space applications and provides them access to system resources owned by *device drivers*.

Drivers for each kind of hardware devices are polymorphic towards the kernel, allowing it to support any combination of hardware components.

Microkernel – examples

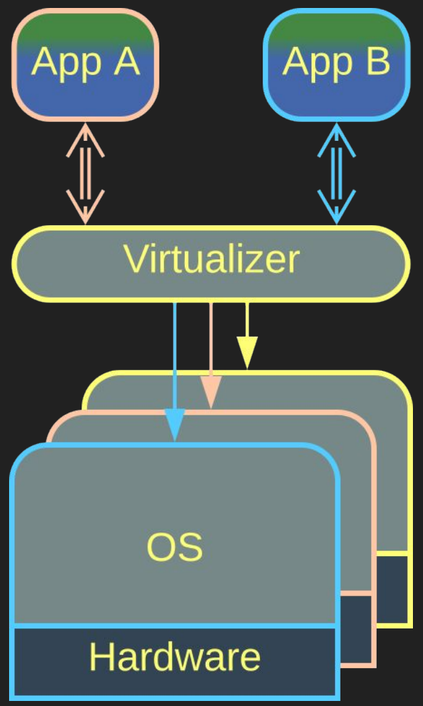


Software Framework

Software frameworks often feature a *Facade* which integrates the framework's internals and provides a stable API to its clients, matching the role of microkernel.

In this case the functionality provided by the framework's internal components is a kind of resource needed by the application.

Microkernel – examples

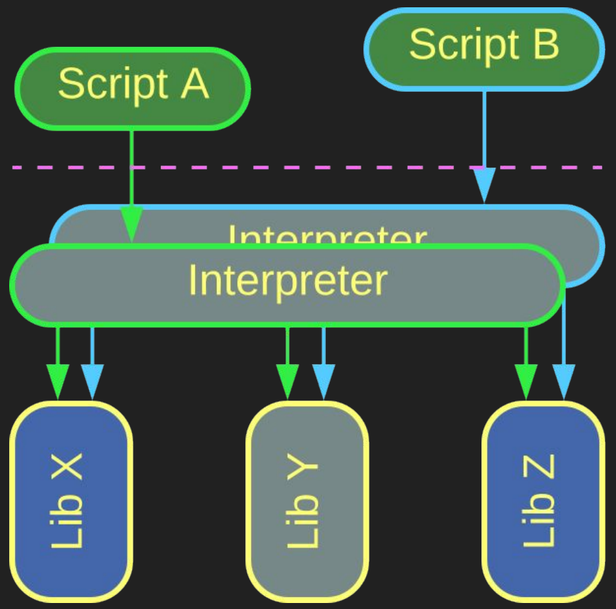


Virtualizer, Distributed Runtime

Virtualizers, *Hypervisors*, and *Distributed Runtimes* use the resources of underlying hardware to run guest applications.

A *Hypervisor* runs on a single computer while a *Distributed Runtime*, such as common actor frameworks, distributes a client application over a pool of servers.

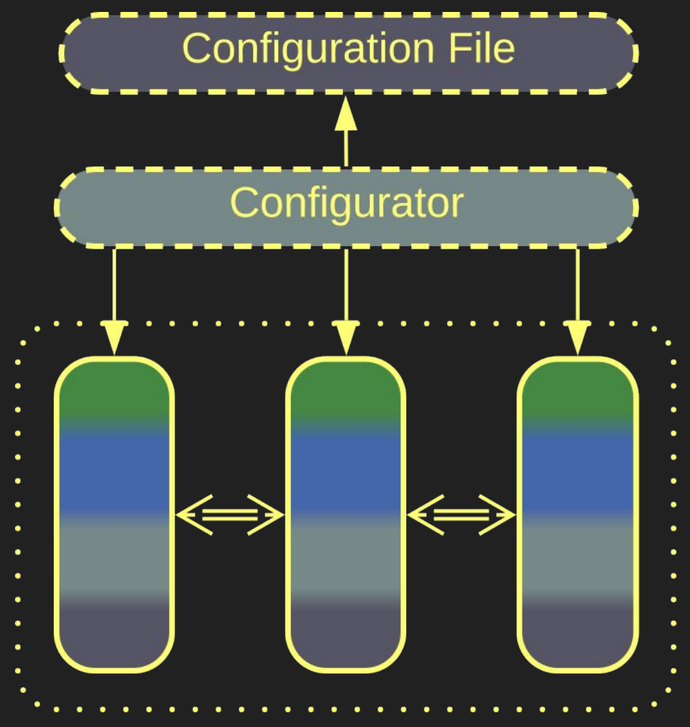
Microkernel – examples



Interpreter

An *Interpreter* executes a client *script* in a sandboxed environment with access to libraries and resources registered with the *Interpreter*.

Microkernel – examples

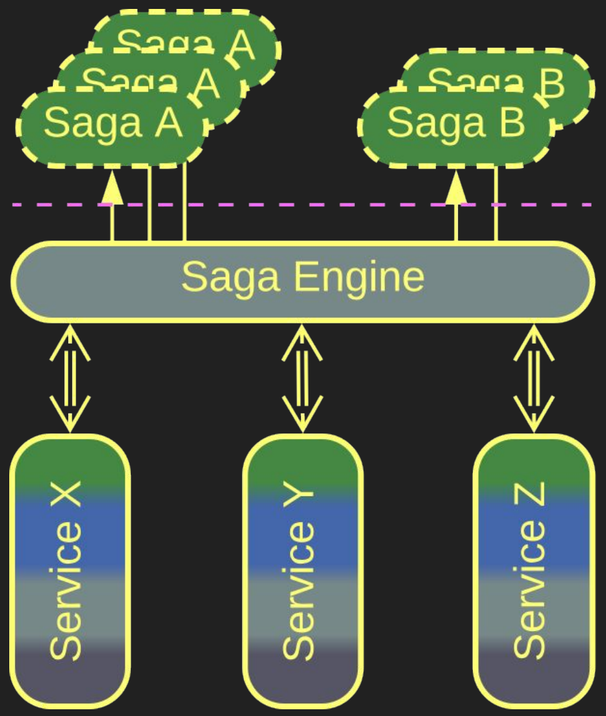


Configurator

A *Configurator* is a short-lived component which runs once on the application's startup. It's task is to set up the application's internal components in accordance to the instructions in the input *configuration file*.

Surprisingly, the *Configurator* is an *Interpreter* for the configuration file, and its creation of the software components is similar to initialization of library classes for use by a script.

Microkernel – examples

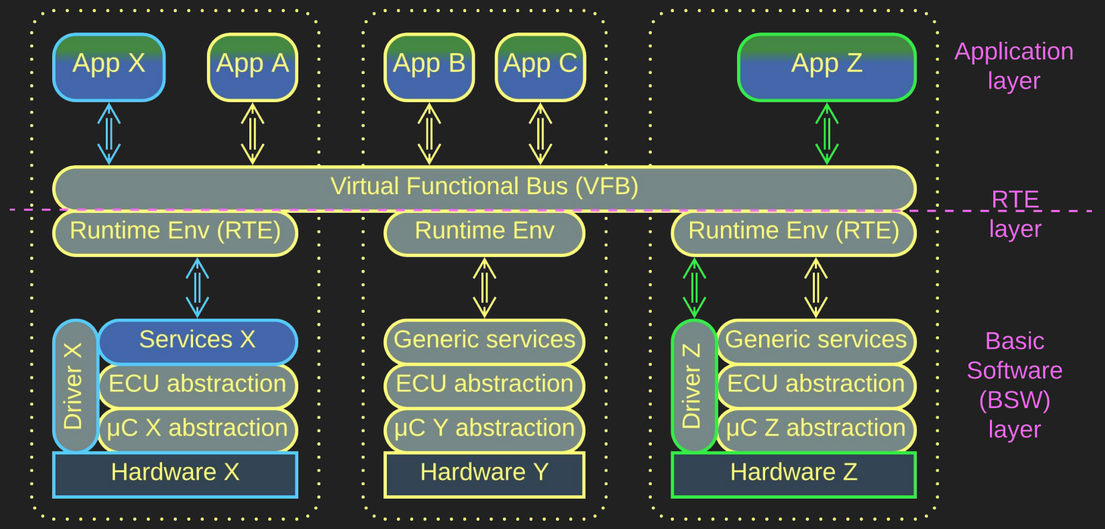


Saga Engine

A *Saga Engine* executes *Sagas* to change states of multiple services in a consistent way.

The *Sagas* are often written in a domain-specific language (DSL) which makes the *Saga Engine* an *Interpreter*.

Microkernel – examples



AUTOSAR Classic

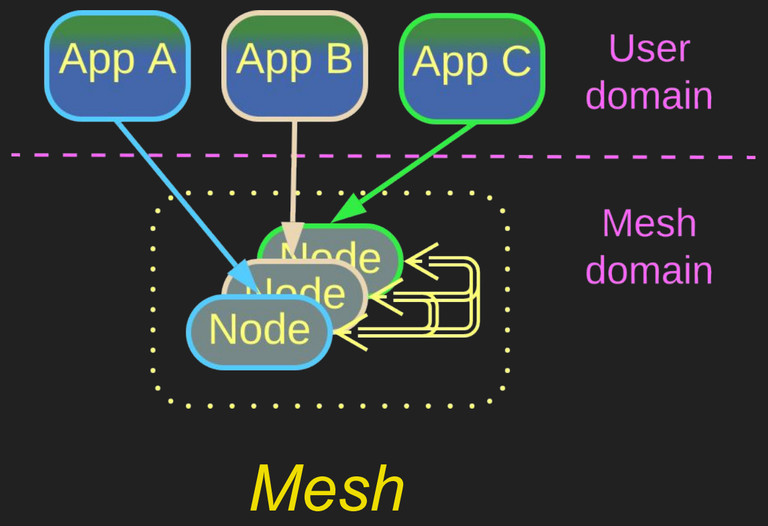
AUTOSAR Classic, the automotive standard, looks exactly like a *Microkernel*, though it was written and marketed as a *Service-Oriented Architecture* (*SOA*).

There is a distributed *Virtual Functional Bus* which connects all the car's components together by allowing every *application* to access any *service* which may run on any of the hundreds chips present in a modern car.

Mesh

Decentralized communication

Mesh – overview



A *Mesh* or *Grid* is a decentralized network. It is extremely fault-tolerant and elastic thanks to dynamic connectivity.

A *Mesh* usually implements a virtual distributed *Middleware* and/or *Shared Repository*.

Mesh is perfect for dynamic scaling and high availability.

It does not work for low latency or security-critical systems.

Mesh – trade-offs

The system is elastic and self-healing with no single point of failure

Mesh implementations are available off the shelf

There is administration and security overhead

Unstable connections degrade performance

The business logic may need to account for unreliable communication of split brain conditions

The Mesh engine is hard to debug

Mesh – structure

Connections between *Mesh* nodes can be:

- *Structured* or *pre-defined*. This kind of *Meshes* provides redundancy but not elasticity as no nodes can be added or removed at run time.
- *Unstructured* or *ad-hoc*. Nodes can be freely added or removed, but the topology of the *Mesh* may be hard to predict, resulting in latency spikes.

Mesh – connectivity

Each *node* can be:

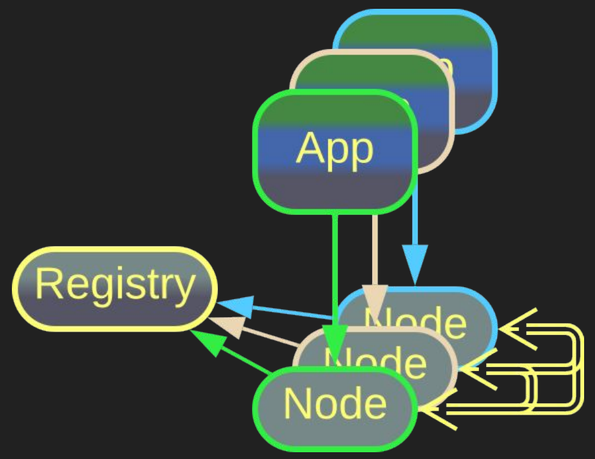
- Connected to every other *node*, making a *fully connected Mesh*. Such *Meshes* are limited in size because of the number of connections, but deliver the best performance and fault tolerance.
- Connected to some other *nodes*. The exact rules for making connections define the internal *topology* of the *Mesh*.

Mesh – layering

The interconnected *nodes* of the *Mesh* can be:

- *Identical* in a *one-layer Mesh*.
- *Specialized* in a *multi-layer Mesh*. In most cases there are *trunk nodes* that maintain the *Mesh*’s *topology* and connectivity, and *leaf nodes* that carry end user applications.

Mesh – examples

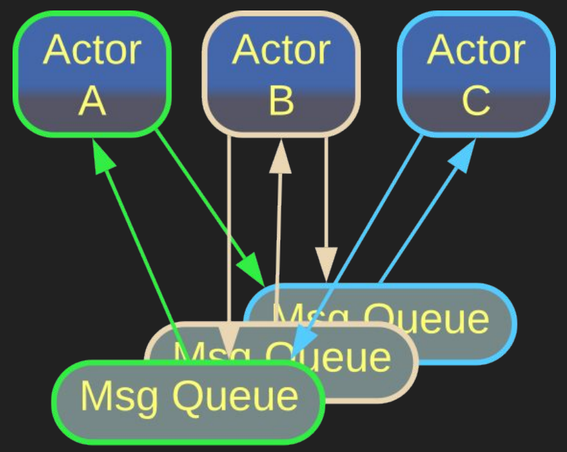


Peer-to-Peer Network

Peer-to-peer Networks share resources (files, CPU time or even Internet access) over unstable connections.

Such a *Mesh* is usually an unstructured (nodes are free to join and leave) two-layer (there are dedicated registrar servers) network that runs on top of the Internet infrastructure and protocols.

Mesh – examples

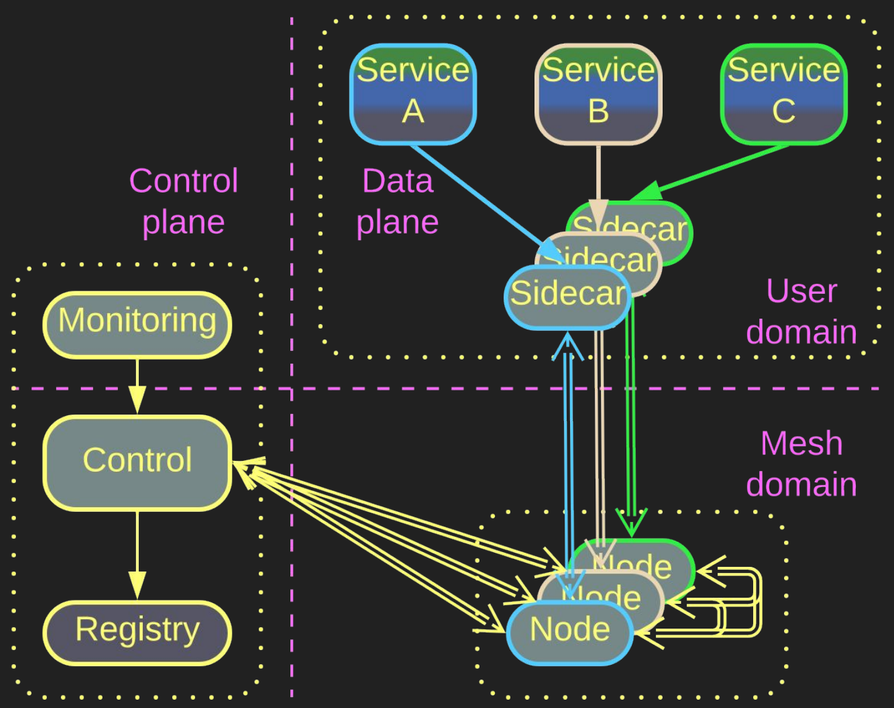


Actors

A system of *Actors* models a fully connected *Mesh* because every *Actor* can send a message to any other *Actor* in the system.

The *message queues* of the *actors*, being their only means of communication, take the role of *Mesh* nodes.

Mesh – examples



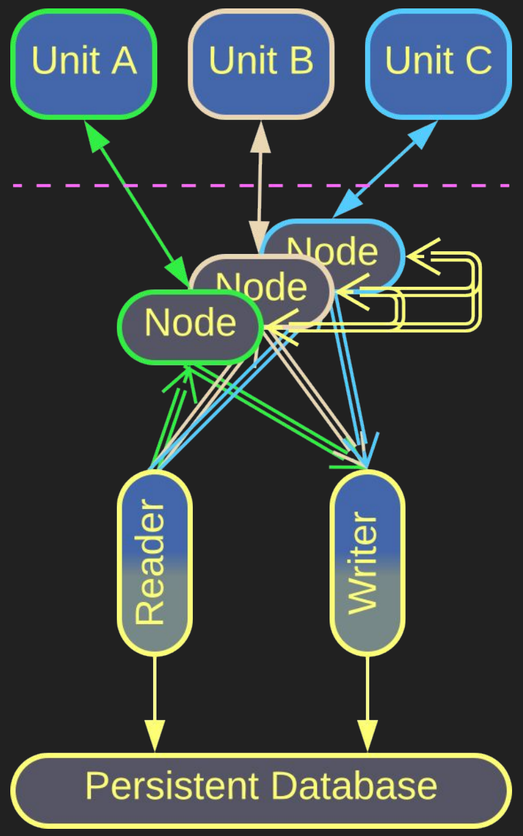
Service Mesh

Service Mesh is a distributed *Middleware* for *Microservices*.

Every instance of a user service is colocated with a *Sidecar* that contains both shared libraries and a *Mesh* node, which maintains the system's connectivity and is governed by a centralized *control* service.

The Mesh nature makes Microservices elastic – new instances can be easily created and added to the network.

Mesh – examples



Space-Based Architecture

Space-Based Architecture is a kind of *Service Mesh* whose *Sidecars* carry nodes of a *Distributed Cache* called *Data Grid*.

The nodes exchange information, ensuring that writes to one node quickly propagate to every other node. As the result, every services has an in-memory copy of the entire system's state.

This architecture provides elasticity at both processing (service instances) and data access level.

Conclusion

The way of patterns

We can modify the internal design of a system component to achieve flexibility with *Plugins* or stability with *Hexagonal Architecture*.

We can share or coordinate resources by employing a *Microkernel*.

We can make our system elastic and fault tolerant by running it in a *Mesh*.

Patterns are like data structures: any system can be implemented with only vectors, or lists, or hash tables. Still, its performance will be better if you use an appropriate algorithm for each data processing task.

Likewise, any business logic can be implemented without patterns, but making prudent use of them will improve both the structure and performance of the code.

Links

This was an overview of patterns revisited in part 5 of *Architectural Metapatterns*. You can:

– read the book online at metapatterns.io

– download it from leanpub.com/metapatterns

The book also contains analytics not covered by these slides.